Розгортання застосунку

Тема 9

Як надійно доставити Spring Boot у production?

Зміст лекції

- Моделі хмарних обчислень IaaS, PaaS, FaaS та провайдери
- Криптографічний захист SSL/TLS, PKI та цифрові сертифікати
- Мережева маршрутизація DNS, IPv6 Dual Stack та тунелювання
- Середовища виконання на VPS, Docker та керування systemd
- Розподілена доставка CDN, патерн Reverse Proxy та Nginx
- Хмарні функції FaaS та архітектура Spring Cloud Function
- Безперебійне оновлення Blue-Green, SSL Bundles та Graceful Shutdown

Спектр середовищ виконання та еволюція хостингу

- Традиційна архітектура Java EE / Jakarta EE вимагала розгортання файлів WAR у важкі сервери застосунків.
- Концепція Fat JAR у Spring Boot перетворила застосунок на самодостатній двійковий артефакт із вбудованим сервером.
- Інфраструктура IaaS надає віртуальні машини з повним контролем над операційною системою та службами systemd.
- Контейнеризація OCI та хмарні моделі PaaS/Serverless стандартизують ізоляцію ресурсів і знижують операційні витрати.

Моделі хмарних обчислень IaaS, PaaS та FaaS

- IaaS (Infrastructure as a Service) надає віртуальні сервери, дискові масиви та віртуальні мережі, де розробник самостійно налаштовує ОС, середовище виконання Java 25 та оновлення.
- PaaS (Platform as a Service) автоматизує життєвий цикл застосунку, самостійно компілюючи код із Git, керуючи масштабуванням і підтримуючи системне середовище.
- FaaS (Function as a Service / Serverless) виконує дискретні обробники подій на вимогу з миттєвим масштабуванням і зменшенням ресурсів до нуля у стані спокою.
- Розподіл відповідальності визначає баланс між повним низькорівневим контролем над системою та швидкістю виведення бізнес-логіки на ринок.

Класифікація сучасних хмарних провайдерів

- Глобальні гіперскейлери (AWS, Google Cloud, Microsoft Azure) пропонують розгалужену мережу дата-центрів і сотні керованих сервісів корпоративного масштабу.
- Інфраструктурні провайдери (Hetzner, DigitalOcean, Linode) надають віртуальні машини з фіксованою та доступною ціною за ресурси та високою швидкістю каналів.
- Платформи розробки (Railway, Render, Fly.io) мінімізують налаштування інфраструктури, надаючи деплой із GitHub, автоматичний SSL та прив'язку доменів.
- Критерії вибору провайдера спираються на бюджет проєкту, вимоги до затримки мережі, простоту супроводу та стандарти комплаєнсу захисту даних.

Зворотне тунелювання локального трафіку через Ngrok та Cloudflare

- Зовнішні сервіси (платіжні шлюзи, хмарні камери, Webhook) не мають прямого маршруту до адреси `localhost`.
- Агенти зворотного тунелювання створюють зашифрований вихідний канал зв'язку до публічного проксі-сервера.
- Команда швидкого публічного тунелю Ngrok: `ngrok http 8080`.
- Безкоштовна альтернатива через хмарний сервіс Cloudflare: `cloudflared tunnel --url http://localhost:8080`.
- Усуває потребу в отриманні білих IP-адрес чи відкритті портів на маршрутизаторі під час розробки.

Фундаментальна роль протоколів SSL і TLS у веб-безпеці

- Протокол HTTP передає дані у відкритому вигляді, створюючи критичні ризики перехоплення чутливої інформації через атаки типу Man-in-the-Middle.
- Протокол TLS (історично SSL) забезпечує надійно зашифрований канал зв'язку за допомогою сучасних криптографічних алгоритмів.
- Гарантує цілісність інформації, запобігаючи несанкціонованій модифікації або підміні мережевих пакетів під час транспортування.
- Забезпечує автентифікацію віддаленого сервера, підтверджуючи клієнту справжність веб-ресурсу та захищаючи від підроблених сайтів.
- Сучасний стандарт TLS 1.3 прискорює встановлення захищеного з'єднання до одного раунду обміну пакетами (1-RTT) та виключає застарілі вразливі шифри.

Інфраструктура відкритих ключів PKI та цифрові сертифікати

- Сертифікат стандарту X.509 зв'язує відкритий ключ шифрування з переліком доменних імен сервера (Subject Alternative Name).
- Ієрархія довіри базується на ланцюжку сертифікатів: кореневий центр сертифікації (Root CA), проміжний центр (Intermediate CA) та кінцевий сертифікат домену.
- Операційні системи, веб-браузери та JVM (сховище `cacerts`) містять попередньо встановлений список довірених публічних центрів сертифікації.
- Протокол ACME дозволяє автоматизувати процес отримання та оновлення безкоштовних виробничих сертифікатів Let's Encrypt.
- Самопідписані сертифікати спричиняють блокування запитів клієнтами через відсутність підпису від визнаного публічного центру довіри.

Архітектурні моделі термінації SSL і TLS

- Термінація SSL на рівні Edge або Reverse Proxy (Nginx, CDN) бере на себе обчислювальні витрати на криптографію.
- Дозволяє розвантажити JVM та спростити внутрішню конфігурацію веб-сервера застосунку.
- Модель наскрізного шифрування (End-to-End TLS) передає трафік зашифрованим безпосередньо у Spring Boot.
- Забезпечує дотримання суворих вимог безпеки корпоративних стандартів (Zero Trust, PCI DSS).

Взаємна автентифікація mTLS та архітектура Zero Trust

- Взаємна автентифікація mTLS (Mutual TLS) вимагає перевірки сертифікатів як сервера, так і клієнта.
- Забезпечує надійну криптографічну автентифікацію між мікросервісами без потреби у статичних токенах.
- Сучасний Spring Boot (починаючи з версії 3.1) надає механізм SSL Bundles для централізованого підключення сховищ ключів.
- Дозволяє динамічно оновлювати сертифікати та сховища довіри без необхідності перезавантаження JVM.

Генерація локальних довірених сертифікатів через mkcert

- Браузери блокують самопідписаний сертифікат SSL попередженнями про загрозу безпеки.
- Утиліта `mkcert` створює персональний локальний центр сертифікації (CA) та додає його до довірених сховищ ОС.
- Команди генерації PKCS12-сховища ключів для локального домену:
  `mkcert -install`
  `mkcert -pkcs12 -p12-file keystore.p12 localhost 127.0.0.1`
- Дозволяє розробляти та тестувати веб-інтерфейси з повноцінною імітацією виробничого HTTPS-середовища.

Налаштування локального SSL у Spring Boot

```
server.ssl.enabled=true
server.ssl.key-store-type=PKCS12
server.ssl.key-store=classpath:keystore.p12
server.ssl.key-store-password=changeit
server.ssl.key-alias=localhost
server.port=8443
```

- Вбудований веб-сервер активує протокол TLS на порту 8443 з використанням сертифіката зі сховища `keystore.p12`.
- Запити за адресою `https://localhost:8443/actuator/health` обробляються без безпекових попереджень у браузері.
- Чутливі паролі та шляхи до сховищ у виробничих конфігураціях передаються виключно через змінні оточення.

Система доменних імен DNS та типи ресурсних записів

- DNS перетворює доменні імена у числові мережеві адреси через багаторівневу ієрархію авторитетних серверів.
- Запис `A` зв'язує доменне ім'я сервісу з 32-розрядною адресою IPv4 хоста.
- Запис `AAAA` адресує запити за 128-розрядною адресою нового протоколу IPv6.
- Запис `CNAME` створює канонічний псевдонім одного доменного імені на інше для хмарних адрес PaaS та CDN.
- Запис `ALIAS` дозволяє створювати псевдонім для кореневої зони домену, уникаючи обмежень стандарту для записів CNAME.
- Параметр TTL (Time to Live) визначає тривалість кешування запису DNS-серверами та швидкість перемикання трафіку при міграції.

Мережева адресація та підтримка IPv6 Dual Stack у Spring Boot

```
server:
  port: 8080
  address: "::"
```

- Доменні імена зіставляються з IP-адресами через DNS: записи `A` для IPv4 та записи `AAAA` для IPv6.
- Прив'язка сервера до адреси `"::"` активує режим Dual Stack, одночасно приймаючи трафік IPv4 та IPv6.
- Усуває проблеми доступності сервісу в сучасних хмарних інфраструктурах із політикою IPv6-only.
- Клієнтські компоненти RestClient коректно опрацьовують IPv6 при обрамленні адреси дужками: `http://[::1]:8080`.

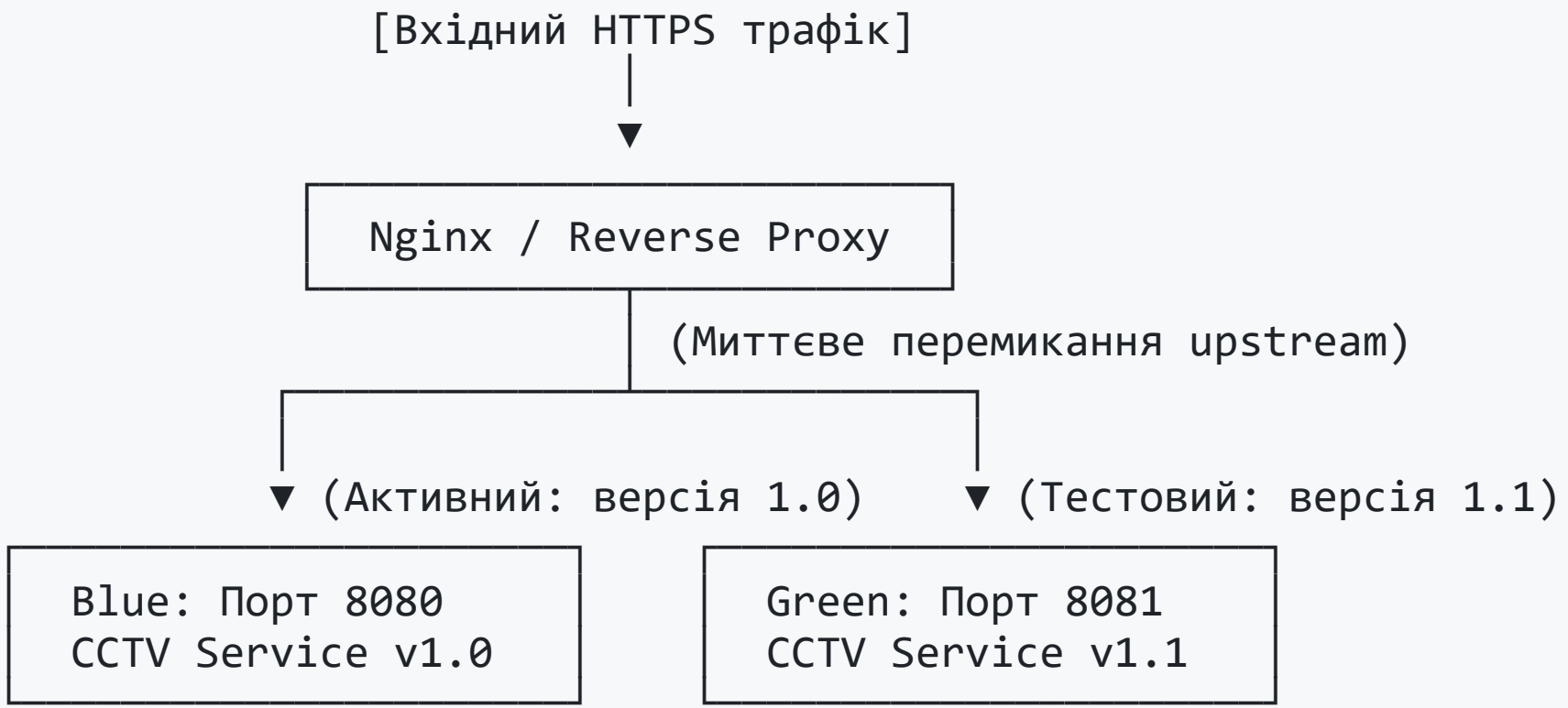
IaaS хостинг та запуск автономного Fat JAR

- Модель IaaS надає віртуальні сервери (VPS) із повним доступом до налаштування Linux ядра та середовища Java 25.
- Плагін `spring-boot-maven-plugin` пакує байткод застосунку та залежності у виконуваний автономний архів JAR.
- Запуск виконується стандартною системною командою: `java -jar cctv-service.jar`.
- Запуск безпосередньо в консолі не підходить для production, оскільки процес завершується при закритті сесії термінала.

Порівняння хостингу на VPS та контейнеризованих середовищ

- Хостинг на VPS передбачає пряме виконання автономного Fat JAR за допомогою системного менеджера процесів systemd.
- Контейнеризація OCI та Docker ізолює застосунок, JVM, нативні бібліотеки та системні залежності у відтворюваному незмінному образі.
- Docker Compose дозволяє однією декларативною конфігурацією локально або на VPS запустити сервіс Spring Boot разом із базою даних PostgreSQL.
- Керовані контейнерні платформи (AWS Fargate, GCP Cloud Run) автоматизують запуск OCI-образів без потреби супроводжувати та оновлювати гостьову ОС.

Схема безперебійного розгортання Blue-Green



- Модель Blue-Green забезпечує паралельне функціонування двох середовищ із миттєвим перемиканням проксі.

Принципи та етапи розгортання Blue-Green

- Підхід Blue-Green розгортає нову версію (Green) паралельно до активної робочої (Blue) на ізольованому порті.
- Перед перемиканням трафіку реліз тестується внутрішніми запитами без жодного впливу на кінцевих користувачів.
- Перемикання виконується атомарно через зміну upstream у Nginx без розриву активних з'єднань (`nginx -s reload`).
- У разі виникнення помилок відкат (Rollback) відбувається миттєво поверненням конфігурації проксі на попередній порт.
- Стратегія вимагає подвійного запасу оперативної пам'яті та процесора для одночасного запуску двох екземплярів застосунку.

Стратегія поетапного оновлення Rolling Updates

```
Фаза 1: [Вузол 1: v1.0]  [Вузол 2: v1.0]  [Вузол 3: оновлення на v1.1...]
Фаза 2: [Вузол 1: v1.0]  [Вузол 2: v1.1]  [Вузол 3: v1.1]
Фаза 3: [Вузол 1: v1.1]  [Вузол 2: v1.1]  [Вузол 3: v1.1]
```

- Стратегія Rolling Updates оновлює екземпляри застосунку послідовно або невеликими групами без переривання сервісу.
- Балансувальник тимчасово виводить цільовий вузол із пулу маршрутизації, оновлює його образ і повертає після перевірки.
- На відміну від Blue-Green, не потребує подвійного запасу обчислювальних ресурсів на хості або в кластері.
- Вимагає суворої зворотної сумісності: база даних і контракти API повинні одночасно підтримувати обидві версії.
- У разі виявлення збою процес оновлення негайно зупиняється, захищаючи

Керування процесами та створення безпечного юніту systemd

```
[Unit]
Description=CCTV Surveillance Core Service
After=network.target

[Service]
Type=simple
User=cctvuser
Group=cctvuser
WorkingDirectory=/opt/cctv
EnvironmentFile=/etc/cctv/cctv.env
ExecStart=/usr/bin/java -jar /opt/cctv/cctv-service.jar
SuccessExitStatus=143
Restart=always
RestartSec=10
ProtectSystem=full
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

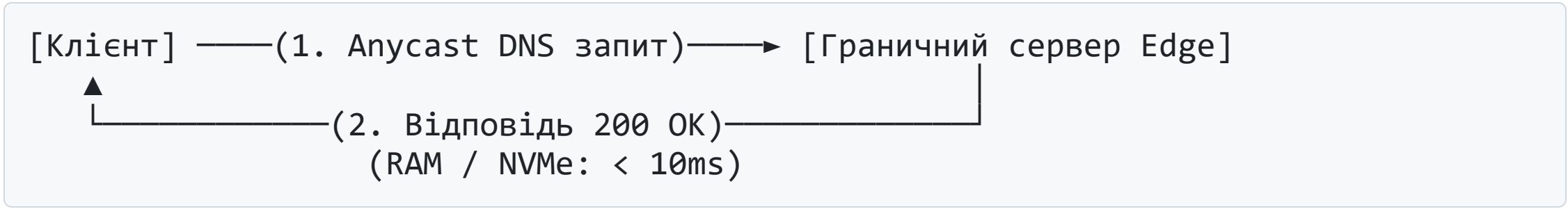
Параметри ізоляції systemd та передача змінних середовища

- Параметр `User=cctvuser` забороняє виконання від імені root, дотримуючись принципу найменших привілеїв.
- Директива `ProtectSystem=full` монтує системні директорії у режимі лише для читання для цього процесу.
- Прапорець `PrivateTmp=true` ізолює тимчасові файли сервісу, запобігаючи несанкціонованому доступу між процесами.
- Файл `EnvironmentFile` завантажує паролі БД та секрети зі сховища з обмеженими правами доступу.

Архітектура та топологія розподіленої мережі CDN

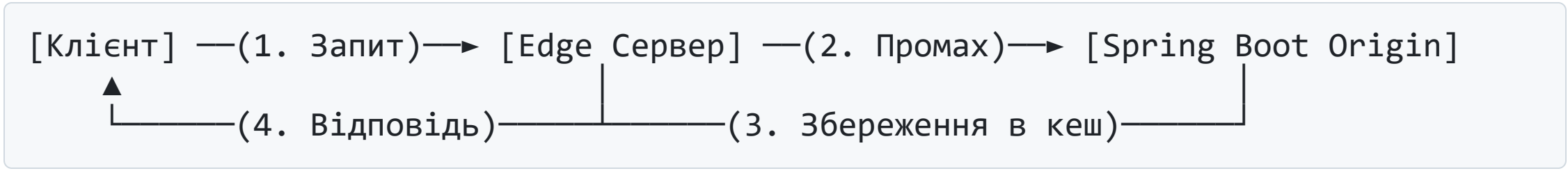
- Мережа CDN складається з географічно розподілених дата-центрів Point of Presence (PoP) на межі мережі (Edge).
- Маршрутизація Anycast транслює однакову IP-адресу з багатьох вузлів, автоматично спрямовуючи клієнта найкоротшим маршрутом BGP.
- Граничні сервери мінімізують затримку передачі даних (RTT), наближаючи обчислення та статичний контент до користувача.
- Застосунок Spring Boot (Origin) ізолюється від прямого публічного навантаження, опрацьовуючи лише унікальні динамічні операції.

Життєвий цикл запиту в CDN та обробка Cache Hit



- Стан Cache Hit виникає, коли запитаний ресурс присутній у пам'яті Edge-сервера і його термін дії не вичерпано.
- Відповідь віддається користувачеві миттєво з найближчого вузла з мінімальною затримкою RTT.
- Звернення до Origin-сервера Spring Boot не відбувається, забезпечуючи максимальну економію ресурсів бекенду.

Життєвий цикл запиту в CDN та обробка Cache Miss



- Стан Cache Miss виникає при першому зверненні до ресурсу, після завершення TTL або після інвалідації.
- Edge-сервер встановлює захищене HTTPS-з'єднання з Origin-сервером Spring Boot та надсилає upstream-запит.
- Отримавши відповідь із заголовками `Cache-Control`, Edge-вузол зберігає копію ресурсу у своєму розподіленому сховищі.
- Механізм Origin Shield об'єднує одночасні промахи у єдиний запит до Origin, запобігаючи перевантаженню бекенду.
- Збережений ресурс стає доступним для всіх наступних клієнтів цього регіону

Керування політиками кешування через заголовки Cache-Control

```
@GetMapping(value = "/api/v1/resources/{id}/data")
public ResponseEntity<ResourceDto> getResourceData(@PathVariable Long id) {
    CacheControl cc = CacheControl.maxAge(Duration.ofMinutes(10)).cachePublic();
    return ResponseEntity.ok().cacheControl(cc).body(resourceService.findById(id));
}
```

- Заголовок `Cache-Control: public, max-age=600` дозволяє CDN кешувати ресурс на 10 хв.
- Директива `immutable` запобігає повторним перевіркам для файлів із версією в назві.
- Приватні API захищаються директивою `no-store, private` для заборони кешування.

Механізм умовних HTTP-запитів з валідатором ETag

```
@GetMapping(value = "/api/v1/resources/{id}/content")
public ResponseEntity<byte[]> getContent(@PathVariable Long id, WebRequest request) {
    String currentETag = resourceService.calculateETag(id);
    if (request.checkNotModified(currentETag)) {
        return null; // Автоматично формує HTTP 304 Not Modified
    }
    return ResponseEntity.ok()
            .eTag(currentETag)
            .body(resourceService.loadContent(id));
}
```

- Валідатор `ETag` (хеш-сума стану ресурсу) перевіряє незмінність даних без передачі самого тіла.
- Метод `request.checkNotModified(etag)` враховує лапки RFC 7232 та автоматично повертає статус HTTP 304 Not Modified.

Фільтрація ETag та інвалідація застарілого кешу

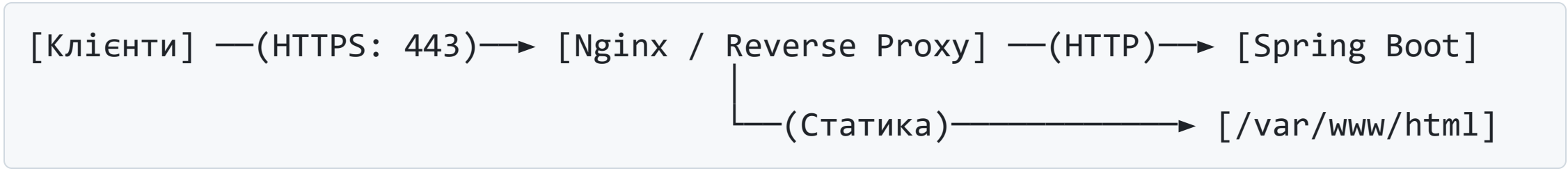
- Фільтр `ShallowEtagHeaderFilter` у Spring Boot може автоматично розраховувати хеші для всіх відповідей.
- Суттєво заощаджує пропускну здатність каналу, віддаючи клієнтам порожнє тіло при збігу версій.
- Інвалідація застарілих даних у мережі CDN ініціюється викликом Purge API провайдера під час оновлення сутності.
- Дозволяє безпечно поєднувати тривале кешування з можливістю миттєвого скидання кешу при зміні даних.

Механізми захисту Origin-сервера від прямого обходу CDN



- Відкритий IP бекенду дозволяє зловмисникам спрямувати DDoS-атаку в обхід CDN.
- Мережевий фаєрвол (UFW) приймає пакети виключно з публічних IP-діапазонів CDN.
- Механізм mTLS або Cloudflare Tunnel повністю виключає прямий доступ до портів.

Роль та архітектурні переваги патерну Reverse Proxy



- Зворотний проксі є єдиною точкою входу, повністю приховуючи внутрішню топологію сервісів.
- Здійснює термінацію SSL/TLS, звільняючи JVM від надлишкових обчислювальних витрат на криптографію.
- Балансує навантаження між екземплярами за алгоритмами Round Robin або Least Connections.
- Забезпечує захисний бар'єр: Rate Limiting, буферизація та роздача статики без участі Java.

Конфігурація зворотного проксі Nginx для Spring Boot

```
upstream spring_backend {
    server 127.0.0.1:8080;
    keepalive 32;
}

server {
    listen 443 ssl;
    listen [::]:443 ssl;
    server_name api.ukma.edu.ua;

    ssl_certificate /etc/letsencrypt/live/api.ukma.edu.ua/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/api.ukma.edu.ua/privkey.pem;

    location / {
        proxy_pass http://spring_backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto https;
    }
}
```

Трансляція клієнтського контексту через X-Forwarded

- Проксі встановлює власне TCP-з'єднання з бекендом, через що Spring Boot за замовчуванням бачить адресу проксі замість клієнта.
- Метод `request.getRemoteAddr()` повертає `127.0.0.1`, що спотворює логи аудиту, системи безпеки та обмеження швидкості.
- Застосунок генерує редиректи з некоректним протоколом `http://` замість публічного `https://` через термінацію SSL на проксі.
- Для збереження автентичного контексту проксі передає заголовки `X-Forwarded-For`, `X-Forwarded-Proto` та `X-Forwarded-Port`.
- Уніфікований стандарт RFC 7239 передає параметри в єдиному заголовку: `Forwarded: for=198.51.100.1;proto=https`.

Налаштування Forwarded Headers Strategy у Spring Boot

```
server.forward-headers-strategy=native
server.tomcat.remoteip.internal-proxies=127\\.0\\.0\\.1|10\\..*
```

- Опція `server.forward-headers-strategy=native` активує клапан `RemoteIpValve` у вбудованому сервері Tomcat.
- Параметр `internal-proxies` обмежує список довірених проксі, блокуючи спроби підробки заголовків зловмисниками.
- Метод `request.getRemoteAddr()` надійно повертає реальну IP-адресу клієнта замість адреси локального проксі.
- Автоматичні редиректи застосунку формують коректні URL з протоколом `https://` та публічним доменним ім'ям.

Хмарна модель PaaS та динамічне призначення порту

```
server.port=${PORT:8080}
spring.main.banner-mode=off
```

- Платформи PaaS (Railway, Render, Fly.io) збирають сервіси безпосередньо з Git репозиторію за допомогою Buildpacks.
- Хмарний маршрутизатор призначає динамічний порт через змінну середовища `PORT`.
- Конфігурація `${PORT:8080}` забезпечує сумісність як із локальним середовищем, так і з PaaS.
- Механізм Relaxed Binding у Spring Boot автоматично пов'язує системні змінні (наприклад, `SPRING_DATASOURCE_URL`) з налаштуваннями.

Архітектура мікрофункцій на базі Spring Cloud Function

- Модель Serverless (FaaS) запускає обчислювальні потужності у відповідь на подію та масштабується до нуля (Scale to Zero).
- Фреймворк Spring Cloud Function усуває прив'язку бізнес-логіки до спеціалізованих SDK хмарних провайдерів (AWS, GCP, Azure).
- Бізнес-логіка реалізується стандартними функціональними інтерфейсами Java пакета `java.util.function`.
- Однаковий скомпільований код працює як звичайний REST-контролер, як обробник черги повідомлень або як хмарна функція.

Функціональні примітиви Supplier, Consumer та Function

```
// Генератор подій (HTTP GET або періодичний відправник у брокер)
@Bean public Supplier<SystemStatusDto> checkStatus() { ... }

// Споживач даних (HTTP POST 202 Accepted або черга Kafka/RabbitMQ)
@Bean public Consumer<AuditLogDto> logEvent() { ... }

// Трансформатор запит-відповідь (HTTP POST 200 OK з результатом)
@Bean public Function<OrderRequest, OrderResult> processOrder() { ... }
```

- Інтерфейс `Supplier<T>` не приймає аргументів і генерує дані, транслюючись у виклики HTTP GET.
- Інтерфейс `Consumer<T>` асинхронно споживає вхідний потік подій без генерації відповіді.
- Інтерфейс `Function<I, O>` перетворює вхідне повідомлення у результат із синхронною відповіддю HTTP 200 OK.

Декларативне створення функціонального біна

```
public record OrderRequest(String orderId, BigDecimal amount, String currency) {}
public record OrderResult(String orderId, String status) {}

@Configuration
public class OrderFunctionConfig {
    @Bean
    public Function<OrderRequest, OrderResult> processOrder() {
        return order -> new OrderResult(order.orderId(), "CONFIRMED");
    }
}
```

- Бізнес-логіка декларується звичайним біном інтерфейсу `Function`.
- Відсутність `@RestController`: повна нейтральність до транспортного протоколу.
- Вхідний потік JSON автоматично десеріалізується у типізований запис Java Record `OrderRequest`.

Інтеграція та серіалізація функціональних бінів

- Результат повернення функції автоматично серіалізується у вихідну JSON-відповідь або повідомлення брокера.
- Spring Cloud Function автоматично реєструє функцію у внутрішньому каталозі `FunctionCatalog`.
- Однаковий код без змін виконується як HTTP REST-ендпоінт, обробник подій черги Kafka чи FaaS-функція.
- Спрощує архітектуру сервісу, фокусуючи розробника виключно на бізнес-перетвореннях даних.

Універсальність виконання та тестування бізнес-функцій

```
@SpringBootTest
class OrderFunctionTest {

    @Autowired
    private Function<OrderRequest, OrderResult> processOrder;

    @Test
    void shouldProcessOrderSynchronously() {
        OrderRequest request = new OrderRequest("ORD-101", new BigDecimal("250.00"), "UAH");
        OrderResult result = processOrder.apply(request);
        assertEquals("CONFIRMED", result.status());
    }
}
```

- Один і той самий код працює як HTTP REST API (`POST /processOrder`), як обробник черги повідомлень або як лямбда-функція FaaS.
- Прозоре юніт-тестування: метод `Function.apply(request)` перевіряється звичайним JUnit без запуску веб-сервера чи мок-об'єктів.

Маршрутизація функцій та конвеєризація Function Composition

```
# Ланцюжок: validateOrder -> processOrder -> sendNotification
spring.cloud.function.definition=validateOrder|processOrder|sendNotification
```

- Конвеєризація через оператор Pipe (`|`) зв'язує функції: вихідний тип попередньої стає вхідним для наступної.
- Ланцюжок типів: `validateOrder` (валідатор DTO) -> `processOrder` (бізнес-логіка) -> `sendNotification` (споживач результату).
- Автоматичний роутинг публікує REST-ендпоінти для всього конвеєра або окремих бінів (`POST /processOrder`).
- Механізм `RoutingFunction` спрямовує вхідні повідомлення на обробники за заголовком запиту.

Адаптери хмарних провайдерів та розгортання в AWS Lambda

- Спеціалізований адаптер `spring-cloud-function-adapter-aws` транслює події AWS (API Gateway, S3, SQS) у виклик біна Java.
- Бізнес-код не містить жодних імпортів бібліотек Amazon чи Google, гарантуючи вільну міграцію між хмарами.
- Повноцінна підтримка юніт-тестування: перевірка логіки через простий виклик `processOrder().apply(request)` без хмарних емуляторів.
- Для мінімізації початкової затримки (Cold Start) застосовують оптимізацію розміру JAR та прогрів екземплярів функцій.

Декларативна конфігурація SSL Bundles у Spring Boot

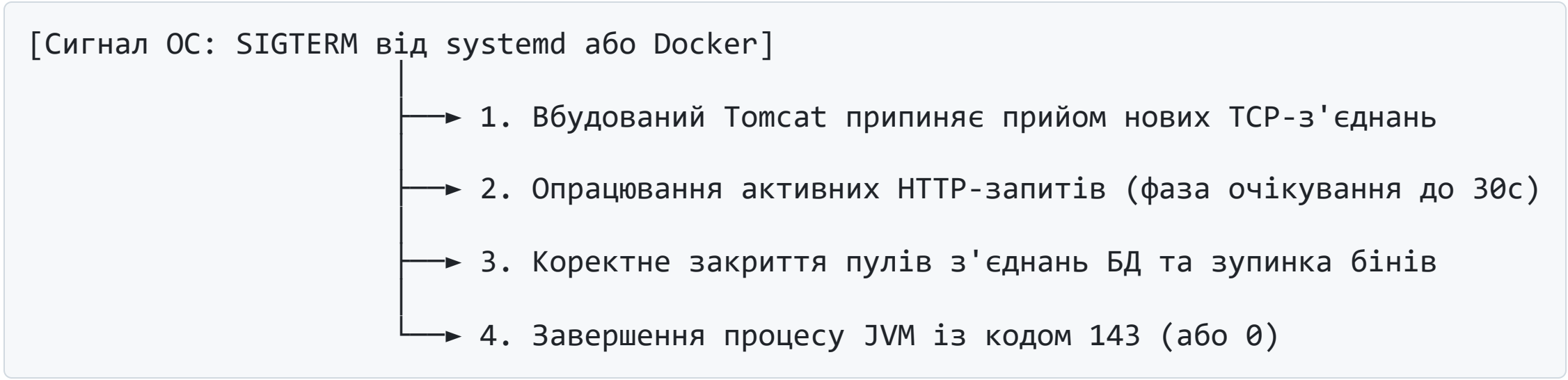
```
spring.ssl.bundle.pem.app-bundle:
  reload-on-update: true
  keystore:
    certificate: "/etc/letsencrypt/live/api.ukma.edu.ua/fullchain.pem"
    private-key: "/etc/letsencrypt/live/api.ukma.edu.ua/privkey.pem"
server.ssl.bundle: "app-bundle"
server.port: 8443
```

- Механізм SSL Bundles уніфікує пряме підключення PEM-сертифікатів і ключів у конфігурації.
- Дозволяє відмовитися від ручного створення та адміністрування сховищ Java KeyStore.

Динамічне перезавантаження сертифікатів без простою

- Директива `reload-on-update: true` запускає фоновий моніторинг файлів сертифікатів на диску.
- При оновленні сертифіката утилітою Certbot сервер перезавантажує TLS-контекст на льоту.
- Повністю усуває необхідність перезапуску процесу JVM при регулярній ротації сертифікатів.
- Забезпечує безперервну доступність захищеного HTTPS-з'єднання для всіх клієнтів сервісу.

Архітектура та послідовність Graceful Shutdown



- Примусове завершення миттєво обриває сокети, генеруючи помилки HTTP 502 Bad Gateway у клієнтів.
- Механізм коректного завершення гарантує безпечне опрацювання активних бізнес-транзакцій.

Життєвий цикл та системні коди завершення JVM

- Після сигналу `SIGTERM` вбудований Tomcat негайно припиняє прийом нових TCP-з'єднань.
- Відповідь HTTP 503 клієнтам повертає зовнішній проксі або балансувальник, якщо вузол виводиться з ротації.
- Код повернення 143 є стандартним у Linux для процесів, зупинених сигналом SIGTERM (`128 + 15`).
- Директива systemd `SuccessExitStatus=143` вказує менеджеру процесів вважати таке завершення штатно успішним.

Налаштування та фази Graceful Shutdown у Spring Boot

```
server.shutdown=graceful
spring.lifecycle.timeout-per-shutdown-phase=30s
```

- Директива `server.shutdown=graceful` активує режим коректного завершення для серверів Tomcat, Jetty та Undertow.
- Параметр `spring.lifecycle.timeout-per-shutdown-phase` задає часовий ліміт для кожної фази зупинки контексту.
- Каскадне вивільнення ресурсів: спершу зупиняється прийом HTTP-запитів, далі фонові задачі, і наприкінці пули HikariCP.
- Таймаут Spring Boot має бути меншим за системні ліміти `TimeoutStopSec` у systemd та таймаут у Docker.

Типові антипатерни конфігурації мережі та зворотного проксі

- Прив'язка публічних сервісів лише до IPv4 `0.0.0.0` там, де вимагається підтримка сучасного IPv6 Dual Stack `"::"`.
- Відсутність конфігурації `server.forward-headers-strategy`, що призводить до втрати IP-адреси клієнта та генерації некоректних посилань HTTP.
- Довіра до всіх проксі без обмеження `internal-proxies` при нативній стратегії, що відкриває можливість підробки заголовків.
- Відкриття незахищеного порту Tomcat 8080 у публічну мережу інтернет замість використання зворотного проксі.

Типові антипатерни безпеки процесів та оновлення

- Запуск процесу в операційній системі від імені суперкористувача `root`.
- Збереження паролів та секретів у відкритому коді замість захищених файлів змінних оточення.
- Відсутність налаштування `server.shutdown=graceful`, що призводить до раптового обриву запитів клієнтів під час оновлення.
- Неконтрольований перезапуск сервісу без попередньої перевірки працездатності та готовності.

Висновки

- Автономний Fat JAR у поєднанні з сервісами systemd забезпечує надійний базовий IaaS хостинг.
- Зворотний проксі у поєднанні з Forwarded Headers гарантує захищену точку входу та збереження клієнтського контексту.
- Моделі PaaS та Serverless на базі Spring Cloud Function суттєво спрощують доставку коду та керування ресурсами.
- Механізм SSL Bundles забезпечує сучасні стандарти безпеки та динамічне оновлення сертифікатів без простою.
- Механізм Graceful Shutdown гарантує безпечне виведення сервісу з експлуатації без втрати активних запитів.

Індивідуальне завдання

1. Створити Spring Boot застосунок та підключити залежність `spring-cloud-function-web`.
2. Реалізувати довільну функціональну бізнес-логіку у вигляді зареєстрованого `@Bean` інтерфейсу `Function`, `Consumer` або `Supplier`.
3. Виконати локальний запуск та перевірити коректність обробки вхідних даних локальним HTTP-запитом через `curl` або Postman.
4. Встановити та налаштувати агент зворотного тунелювання Ngrok для прокидання порту локального веб-сервера: `ngrok http 8080`.

Індивідуальне завдання

1. Зафіксувати згенеровану публічну HTTPS-адресу з виводу термінала або панелі керування Ngrok.
2. Надіслати тестовий запит із зовнішньої мережі на отриману публічну адресу за маршрутом створеної функції.
3. Перевірити проходження запиту в логах застосунку та зберегти знімок екрана з успішною відповіддю HTTP 200 OK.
4. Завантажити на DistEdu архів розробленого проєкту разом із підтвердженням доступності функції через Ngrok.

Групове завдання

1. Підготувати командний застосунок до розгортання в цільовому оточенні: на сервері VPS (через юніт systemd) або в ізольованому Docker-середовищі.
2. Забезпечити автоматичний запуск сервісу, ізоляцію прав користувача та передачу конфігураційних параметрів через змінні середовища.
3. Налаштувати політику безпечного завершення `server.shutdown=graceful` для запобігання втраті активних запитів при оновленні версій.

Групове завдання

1. Забезпечити зовнішню мережеву доступність розгорнутого застосунку з публічної мережі Інтернет.
2. Провести тестування зовнішнього доступу та переконатися в успішній обробці вхідних запитів сервісом.
3. Підготувати короткий звіт про обрану архітектуру розгортання з файлами налаштувань та завантажити результати на DistEdu.